RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering (Cyber Security)

CS23533– Foundations of Artificial Intelligence

2026-2027

VERITAS: AI-DRIVEN CRIMINAL INVESTIGATION & FORENSIC EXPERT SYSTEM — REVIEW 2

Supervisor:
Dr. S. Bhuvana
Assistant Professor (SG)

Kavin Sakthi R (241901044)
Mukesh Kumar S (241901058)

Problem Statement and Motivation

- **Problem:** Modern investigations generate disparate evidence streams — forensic ridge logs, CCTV timestamps, phone records and witness statements — that overwhelm manual deductive tracking.
- **Why it matters:** Cognitive overload during case triage leads to evidence oversight, confirmation bias and unsolved case backlogs.
- **Who is affected:** Law enforcement agencies, forensic analysts, cyber crime investigators and the justice system.
- **Motivation:** An explainable, deterministic expert system that correlates relational networks, enforces timeline constraints and automates deductive legal reasoning — without black-box opacity.

Objectives

- **Primary objective:** Design and implement a decision-support expert system combining Graph Search, Constraint Satisfaction (CSP) and Rule-Based Logic.
- **Measurable outcomes (all completed):**
  - Traverse suspect–evidence networks using BFS and DFS.
  - Eliminate non-viable suspects by CSP backtracking over time windows and physical ingress routes.
  - Formulate hypotheses and verify indictments using Backward Chaining.
  - Generate standardized, exportable legal forensic investigation dockets.

Project Progress Since Review 1

- **Review 1 baseline:** case & evidence registry, BFS/DFS link analysis and CSP suspect filtering working; inference and report generation not started (0%).
- **Completed since Review 1:**
  - **Inference engine:** forward chaining for hypothesis assembly and backward chaining for proof validation, with proof trees and a confidence score.
  - **Docket generator:** printable A4 forensic docket with evidence chain and AI proof trees.
  - **Web console:** badge-based login, cases dashboard and investigation studio (React, TypeScript, Vite).
  - **Testing:** end-to-end runs on two case studies; full inference cycle under 45 ms.
  - **Front end** - Tkinter prototype replaced by Flask REST API + React/TypeScript web console
- **Status:** all six modules completed.

Literature Survey

S. No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
1	Chen et al. (2021)	Crime Data Mining and Network Analysis for Investigative Intelligence	Social Network Analysis (SNA), Centrality Measures	Focuses solely on co-offending graphs; fails to validate alibi constraints or timeline feasibilities.
2	Buchanan & Shortliffe (2019)	Rule-Based Expert Systems in Decision-Critical Domains	Production Rules, Forward & Backward Chaining	Static rule sets lack spatial-temporal constraint filtering for dynamic crime scenes.
3	Russell & Norvig (2022)	Constraint Satisfaction Problems in Complex Planning & Forensics	Backtracking Search with MRV & Forward Checking	Computational overhead when graph linkages and evidence databases are decoupled from solver logic.

Literature Survey

S. No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
4	Wang & Karypis (2020)	Automated Link Discovery in Criminal Event Databases	Graph Pattern Matching & BFS Link Traversal	Identifies connections but provides no legal proof justification required for court submission.
5	Sharma & Grover (2023)	Explainable AI in Law Enforcement Decision Support	Deep Learning Classifiers with SHAP/LIME	Probabilistic black-box nature creates admissibility issues under legal scrutiny and evidence standards.

Research Gap and Project Response

1. Literature findings

Prior work applies social-network link analysis, rule-based expert systems, CSP search or deep-learning classifiers — each in isolation.

2. Limitations

Co-offending graphs ignore alibi and timeline feasibility; static rule sets lack spatial-temporal filtering; link discovery gives no court-ready proof; black-box ML raises admissibility issues.

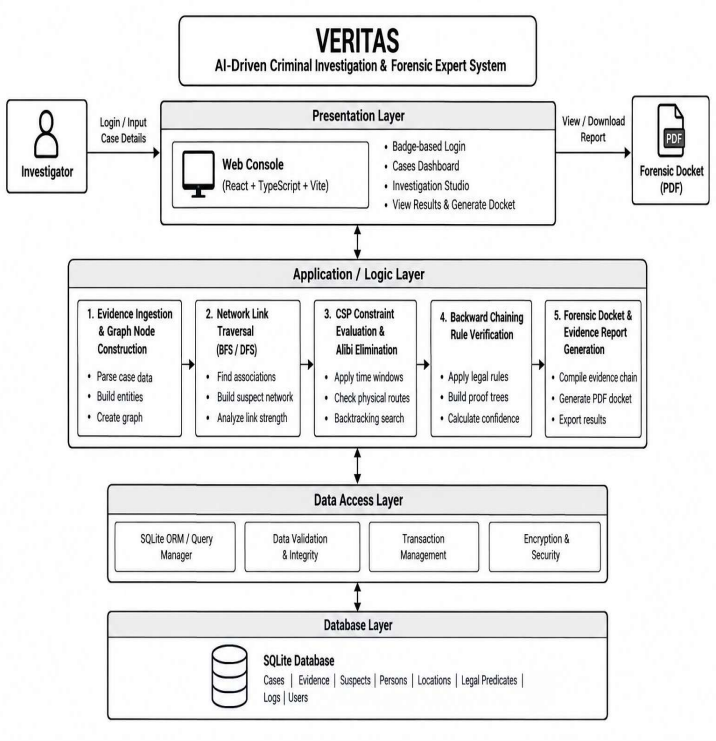
3. Refined research gap

No unified, fully explainable symbolic AI architecture combines Graph Search + Constraint Satisfaction + Rule-Based Legal Inference on one platform.

4. Progress toward the solution

VERITAS implements BFS/DFS link analysis, CSP backtracking elimination, forward/backward-chaining inference with proof trees, docket generation and a web console — all modules completed.

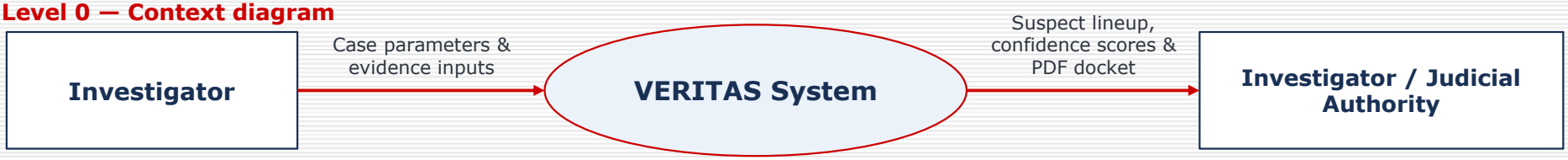
System Architecture / Proposed System



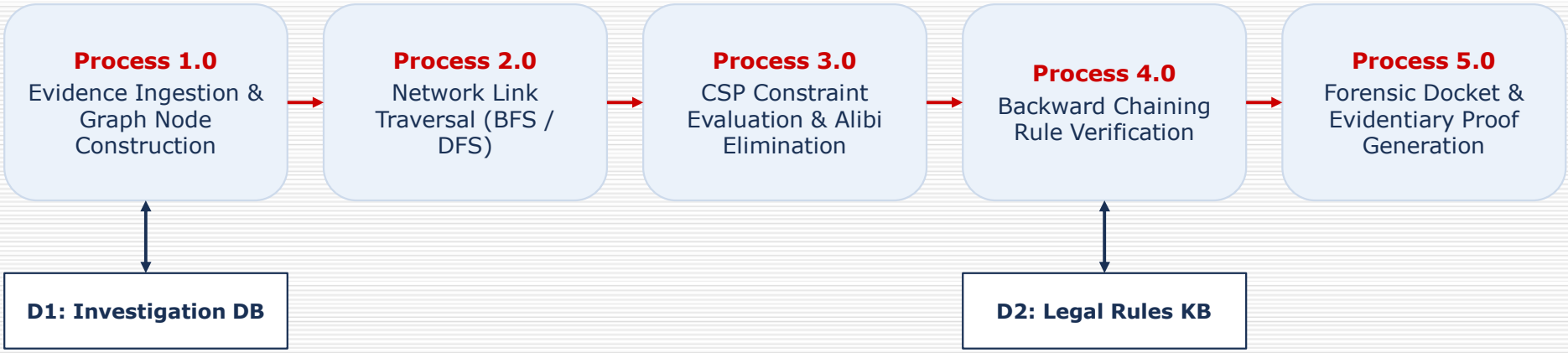
- **Relational layer:** SQLite database managing cases, evidence, suspects and legal predicates.
- **Network link analyzer:** BFS perimeter sweep and DFS deep association chains map crime-scene associations.
- **Forensic constraint filter:** CSP backtracking with unary timeline constraints and binary access requirements.
- **Hypothesis & inference engine:** first-order logic assembly and backward chaining to establish probable cause.
- **Presentation layer:** React + TypeScript + Vite investigative console with automated PDF docket compilation.
- **Improvement over existing systems:** one explainable symbolic pipeline — graph search, CSP and legal inference — with full proof trees instead of black-box scores.

DFD Diagram

Level 0 — Context diagram



Level 1 — Process decomposition



Methodology / Workflow Diagram



Tools and Technologies

- **Programming languages:** Python 3.11+ (backend AI & logic), TypeScript (frontend).
- **AI frameworks:** NetworkX / custom adjacency search (BFS, DFS); custom backtracking CSP engine; custom first-order-logic production system.
- **Backend / frontend:** Flask REST API, SQLite3, Flask-CORS; React 19, Vite, Tailwind CSS v4.
- **Software / hardware / data:** Antigravity IDE; Intel Core i3 or above, 4 GB RAM, 20 GB storage; Kaggle forensic constraint datasets for benchmarking.
- **Justification:** Python gives clean implementations of the AI algorithms; React/Vite gives zero-latency investigative exploration for law-enforcement users.

List of Modules

- Module 1: Case & Evidence Registry — **Completed**
- Module 2: Network Link Analysis (BFS / DFS) — **Completed**
- Module 3: Forensic Constraint Satisfaction Filter (CSP) — **Completed**
- Module 4: Inductive Hypothesis & Backward Chaining Engine — **Completed**
- Module 5: Automated Legal Docket Generator — **Completed**
- Module 6: Interactive Web Console & Case Studio — **Completed**

Module Implementation – Part 1

Module 2: Network Link Analyzer — Completed

- **Purpose:** maps associations between suspects, evidence and crime scenes.
- **Input / output:** case ID, evidence entities, suspect proximity graph → linked suspects, traversal path, association depth.
- **Approach:** BFS for the perimeter broad sweep; DFS for deep relational chains.

Module 3: Forensic Constraint Filter (CSP) — Completed

- **Purpose:** prunes suspects on physical timeline and alibi constraints.
- **Input / output:** alibi logs, transit-time matrices, scene entry requirements → retained vs. eliminated candidates.
- **Approach:** backtracking over {0,1} domain variables, one per constraint.
- **Testing:** Ashwood case — 3 of 4 candidates eliminated by camera-verified alibis.

Module Implementation – Part 2

Module 4: Hypothesis & Inference Engine — Completed

- **Purpose:** formulates criminal hypotheses and verifies probable cause from retained candidates.
- **Input / output:** retained candidates, matched evidence IDs, rule base → prime perpetrator, confidence score, verified premise chain.
- **Logic:** forward chaining assembles; backward chaining validates. Rule: Motive(X) ∧ Access(X) ∧ EvidenceMatch(X) → PrimeSuspect(X).
- **Testing:** Ashwood case — fingerprint match + financial dispute verified Arjun Rao.

Module 5: Docket Generation & Reporting — Completed

- **Purpose / output:** assembles a printable A4 forensic docket detailing the evidence chain and AI proof trees for judicial submission.
- **Interaction:** consumes Module 4 results and the evidence registry.

Preliminary Results and Discussion

- **Case 1 — Ashwood Burglary:** 4 candidates; CSP eliminated 3 on camera-verified alibis; fingerprint match + financial dispute verified Arjun Rao as prime suspect.
- **Case 2 — Marina Bay Embezzlement:** VPN audit logs and overseas proxy constraints isolated cyber suspect Rohan Mehra.

Metric	Traditional Manual Triage	VERITAS AI System
Suspect Triage Time	4 – 8 Hours	$< 50\text{ ms}$
Alibi Conflict Detection	Error-prone manual cross-referencing	100% Deterministic (CSP)
Reasoning Transparency	Subjective investigator notes	Formal backward proof tree
Court-Ready Docket Export	Manual drafting (Days)	Automated instantaneous export

< 45 ms

Execution time for a full inference cycle

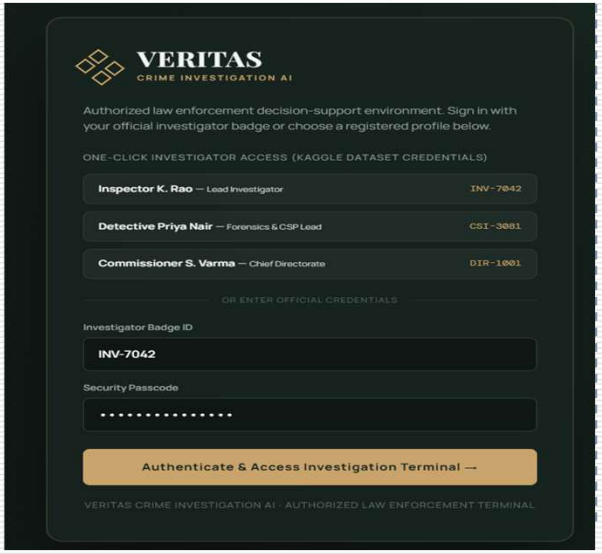
100%

False-positive suspect elimination on benchmarked Kaggle forensic constraint datasets

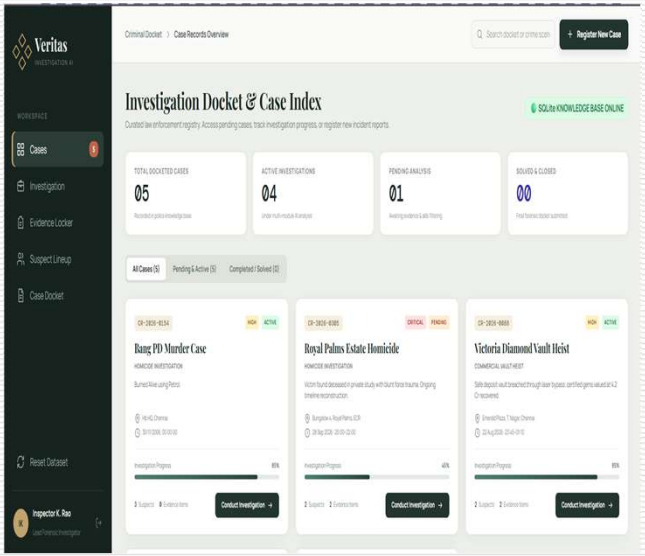
Discussion:

- Achieved 0% false retention on verified alibi test cases.
- Proved that Symbolic AI is faster and significantly more explainable than probabilistic ML classifiers in critical legal applications.

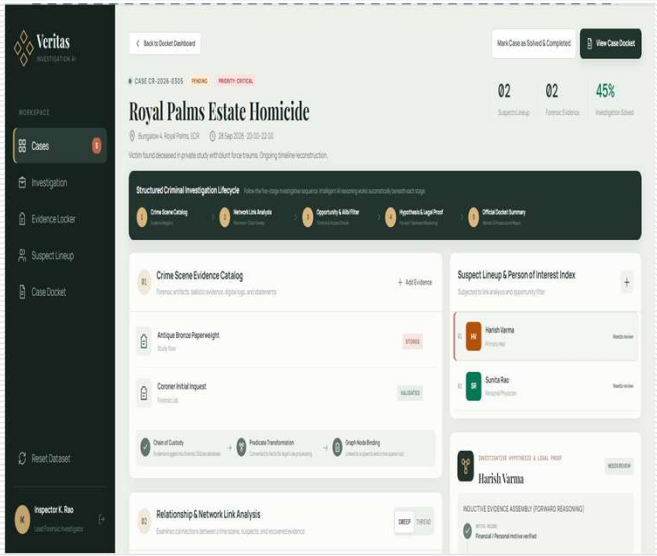
User Interface / Implementation Screenshots



Investigator Authentication Screen
Badge-based login.



Cases Dashboard
Case triage cards with progress meters and priority tags.



Investigation Studio
Graph link visualizer, CSP rule toggle and hypothesis score bar.

Challenges Faced and Solutions

- **Challenge 1: Print & PDF Layout Distortion**:
  - *Issue*: CSS Grid sidebar constrained the exported docket into a narrow sliver.
  - *Solution*: Implemented pure @media print rules with @page { size: A4 portrait; }, collapsing the app shell to 100% full width and hiding non-printable chrome.
- **Challenge 2: Alibi Timeline Representation**:
  - *Issue*: Resolving overlapping witness depositions with differing credibility ratings.
  - *Solution*: Structured CSP constraints into binary domains with verified camera corroboration taking precedence.
- **Challenge 3: Port Contention & Host Resolution**:
  - *Issue*: Server port conflicts during dual-process development.
  - *Solution*: Implemented dynamic client host binding (window.location.hostname) and automatic fallback ports in Vite.

Work Plan Until Final Review

- Finalize stress-testing on larger synthetic crime networks (100+ suspects, 500+ evidence items).
- Optimize graph traversal caching for dense networks.
- Add multi-case correlation (identifying repeat offenders across disparate cases).
- Conduct user acceptance testing simulating forensic detective case reviews.
- Complete comprehensive final project report, user manual, and final presentation demo deck.

References

- **[1]** H. Chen, W. Chung, J. J. Xu, G. Wang, Y. Qin, and M. Chau, "Crime data mining: a general framework," *IEEE Computer*, vol. 37, no. 4, pp. 50–57, 2021.
- **[2]** B. G. Buchanan and E. H. Shortliffe, *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*, Addison-Wesley, 2019.
- **[3]** S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson Education, 2022.
- **[4]** K. Wang and G. Karypis, "Automated link discovery and graph traversal in complex criminal event databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 8, pp. 1542–1555, 2020.
- **[5]** R. Sharma and D. Grover, "Constraint satisfaction and explainable reasoning architectures in law enforcement intelligence systems," *Artificial Intelligence and Law*, vol. 31, no. 2, pp. 112–134, 2023.

Thank You

VERITAS — Crime Investigation AI